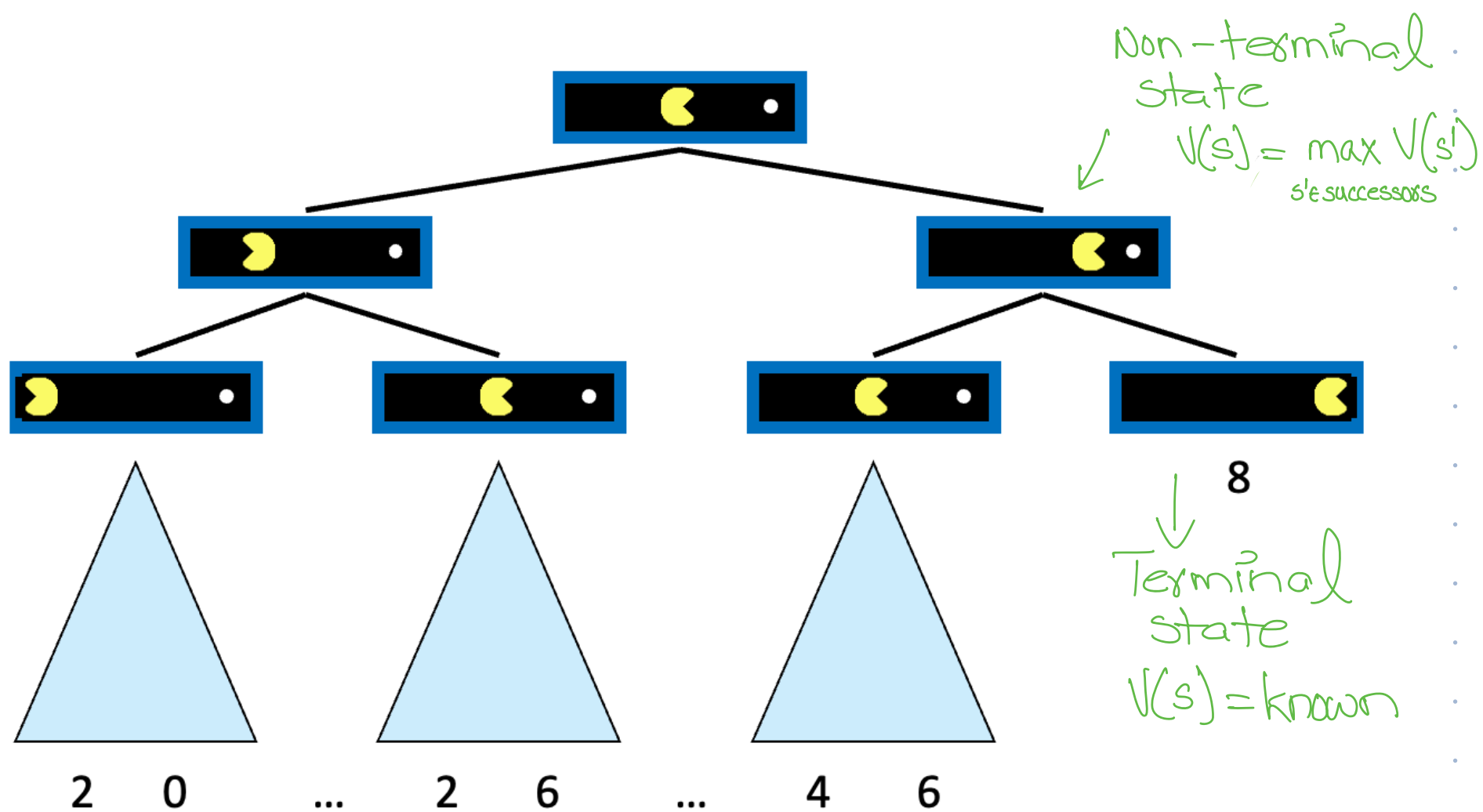


Value of a state

Best achievable outcome (utility) from that state.

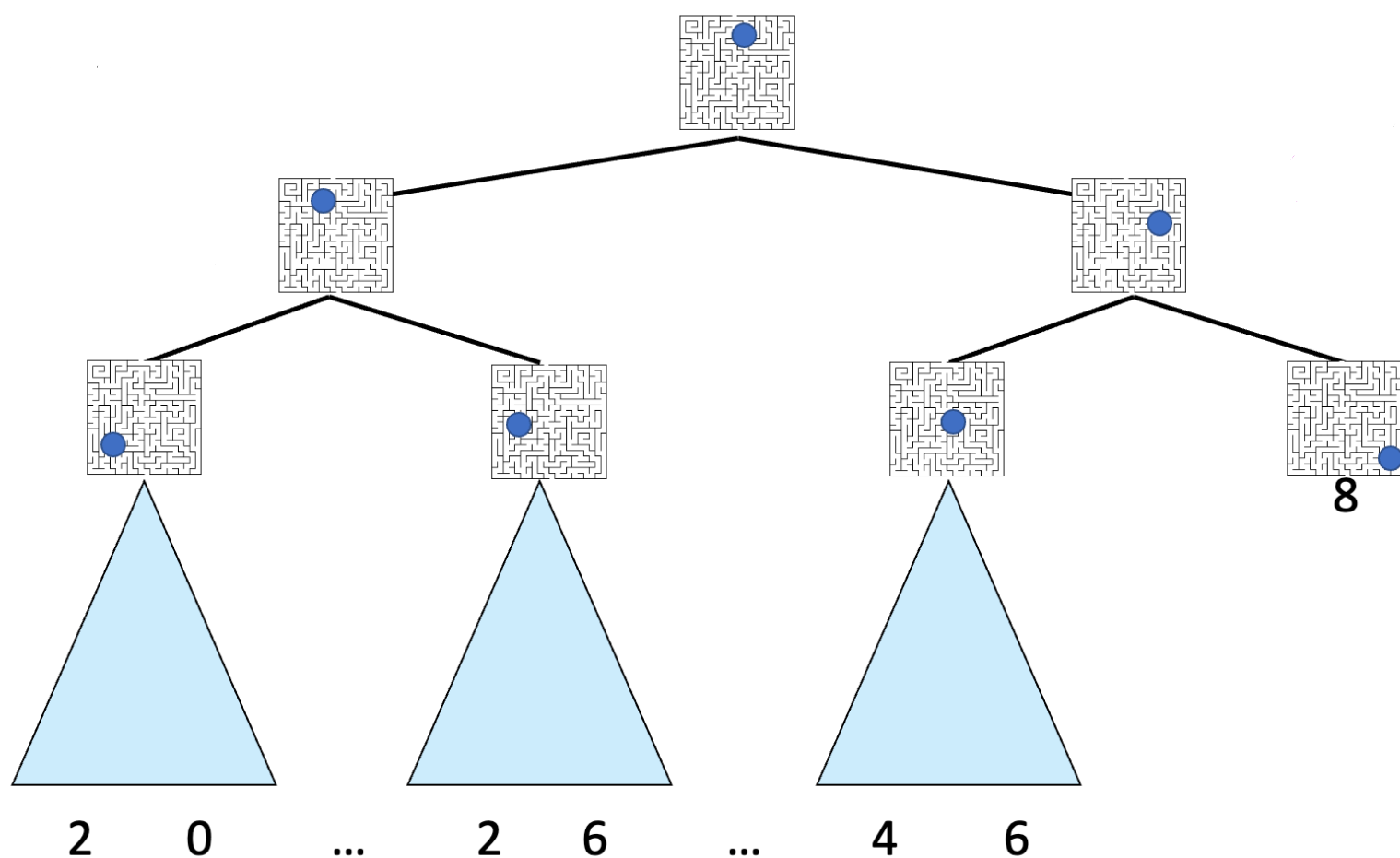


How to model multi-agent collaborative problems?

Idea 1:

Each agent plans their own action separately from others

Choose the best action for each agent independently



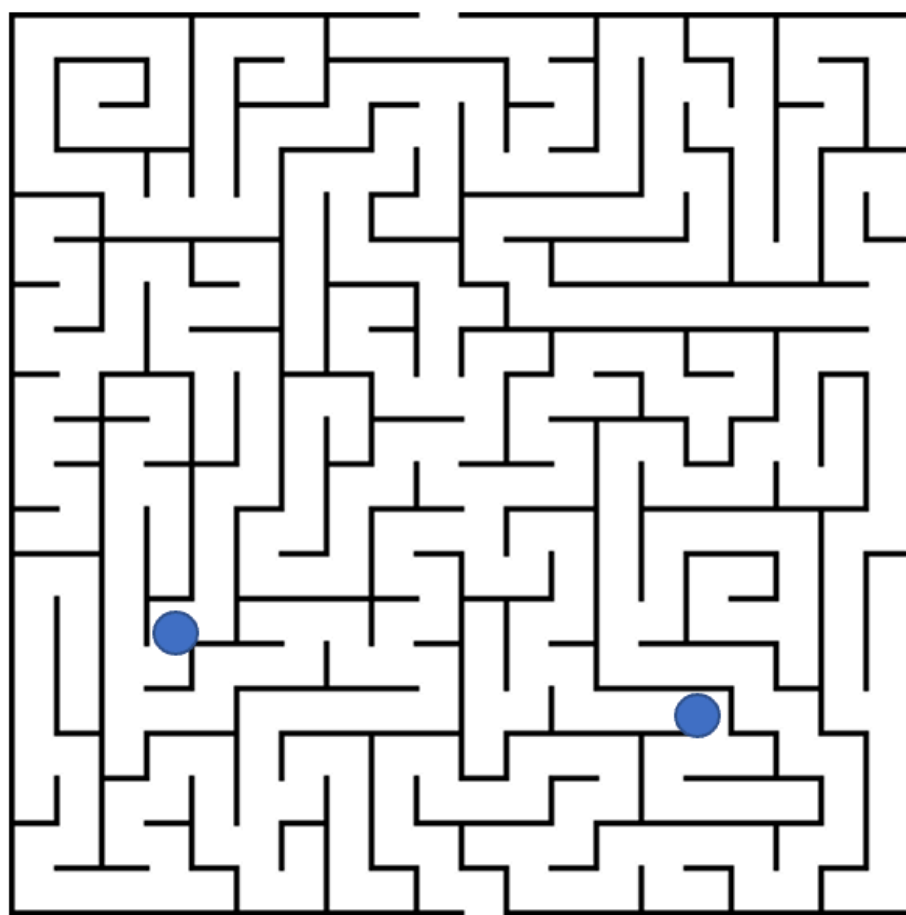
Idea 2:

Combine the states & actions of N agents

Search through all combinations of all agents' states & actions

Think of one brain controlling many agents

$$S_K = (S_K^A, S_K^B)$$

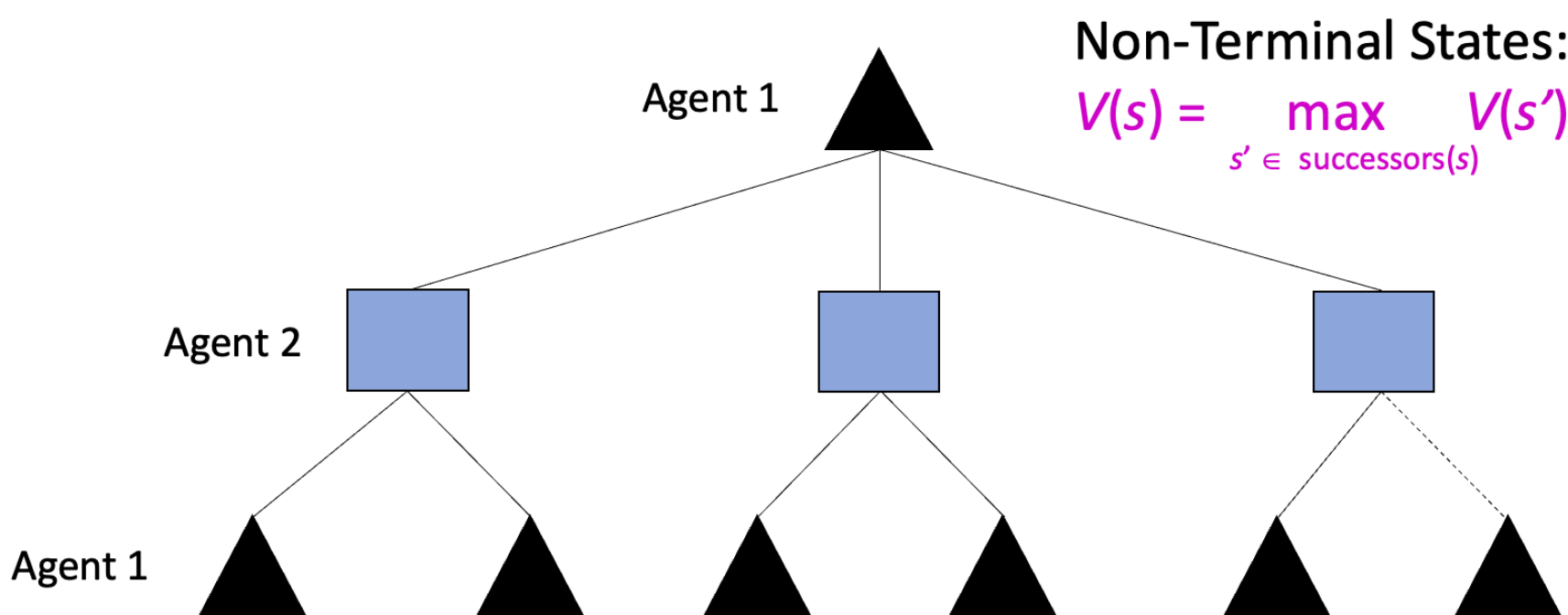


Idea 3:

Alternate searching one agent at a time.

Search one agent's action from a state, search the next agent's actions from those resulting states & so on.

Choose the best cascading combination of actions



Standard games:

↳ They are deterministic, observable, two-player, turn-taking, zero-sum.

Game formulation

↳ Initial state: s_0

↳ Players: $\text{Player}(s)$ indicates whose move it is

↳ Actions: $\text{Actions}(s)$ for player on move

↳ Transition model: $\text{Result}(s, a)$

↳ Terminal test: $\text{Terminal-Test}(s)$

↳ Terminal values: $\text{Utility}(s, p)$ for player p

Zero-Sum Games

↳ Agents have opposite utilities

↳ Pure competition
one maximizes, the other minimizes

General Games

↳ Agents have independent utilities

↳ Cooperation, indifference, competition, shifting alliances, and more are all possible.

Tic-tac-toe Game Tree:

↳ Search one agent's actions from a state, search the competitor's actions from those resulting states.

↳ This is a zero-sum game.

The best action for X is worst action for O & vice-versa.

↳ Instead of taking max utility at every level, alternate max & min.



MAX (X)



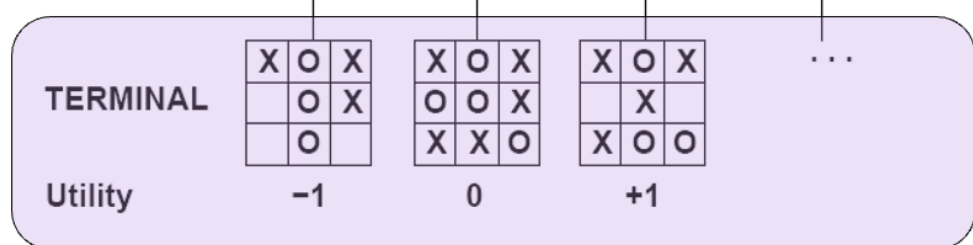
MIN (O)



MAX (X)



MIN (O)

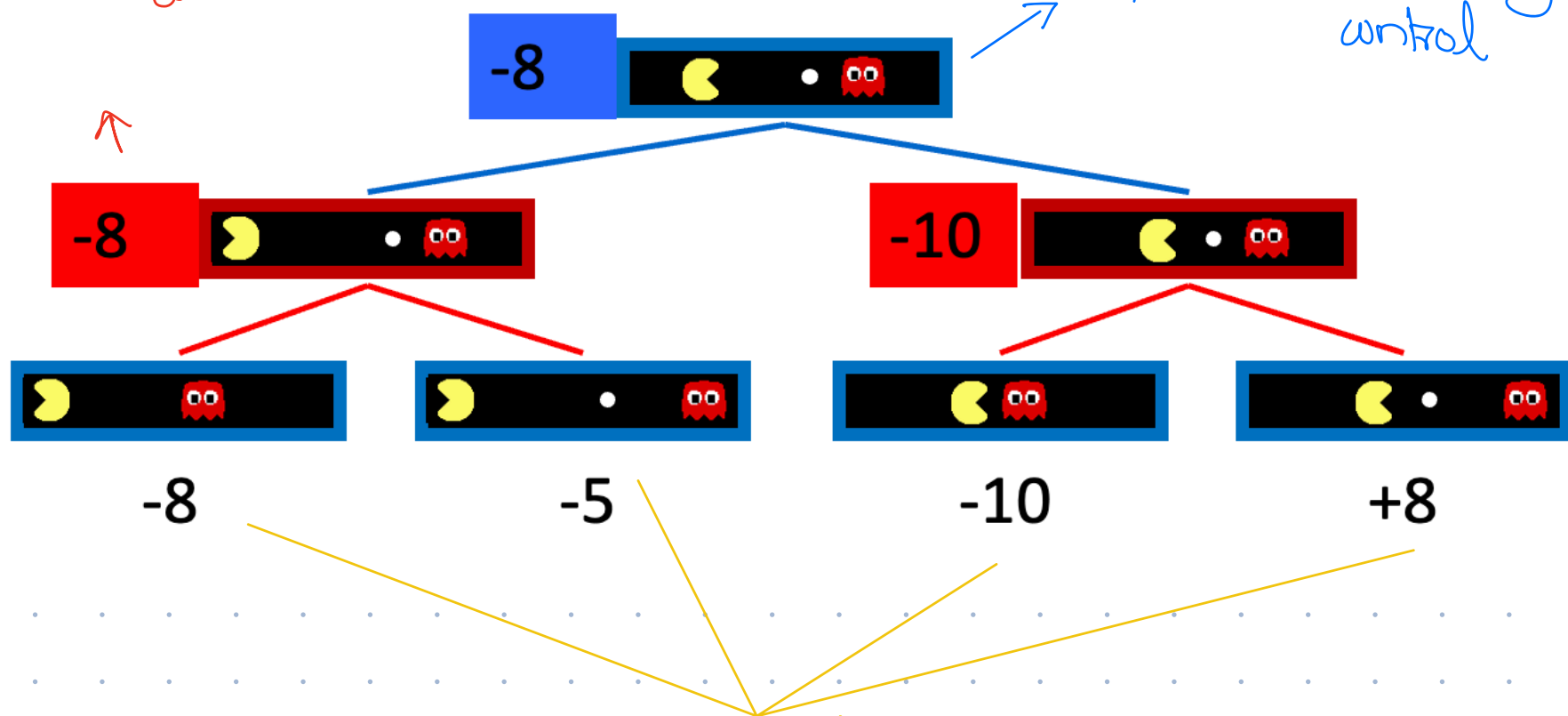


max nodes under agent's control
 $V(s) = \max_{s' \in \text{successors}} V(s')$

min nodes under opponent's control
 $V(s) = \min_{s' \in \text{successors}} V(s')$

Example,

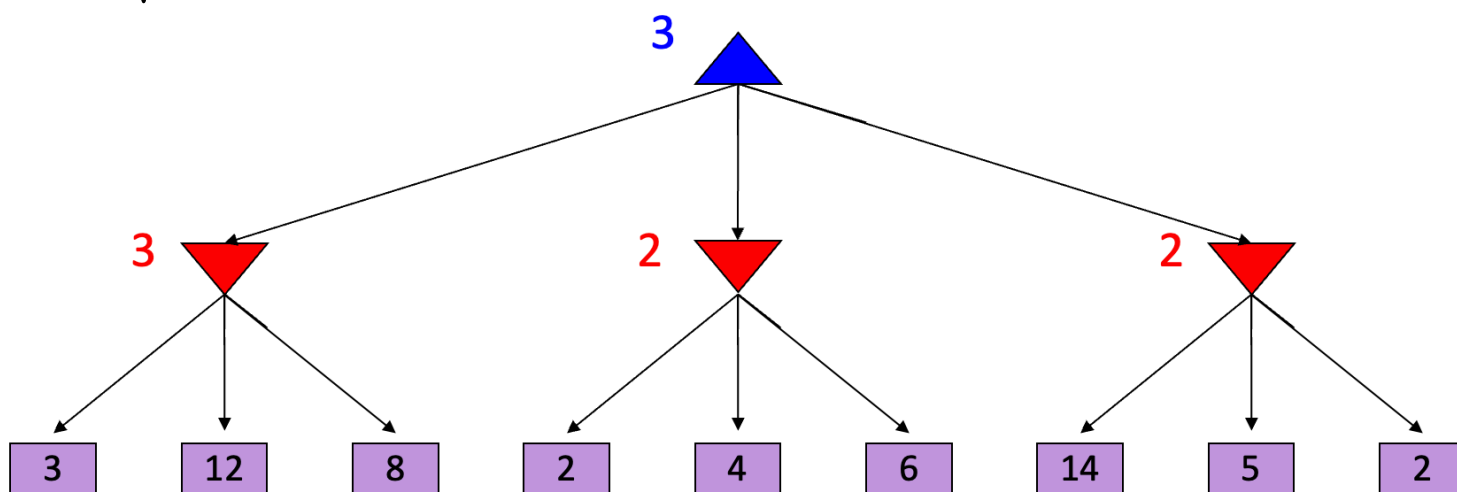
MIN nodes: Under opponent's control



MAX nodes: under agent's control

Terminal states.
 $V(s) = \text{known}$

Another example,



Mini-max implementation

function minimax-decision(s) returns action

return the action a in $\text{Actions}(s)$ with the highest $\text{min-value}(\text{Result}(s,a))$

function max-value(s) returns value

if Terminal-Test(s) then return Utility(s)

initialize $v = -\infty$

for each a in $\text{Actions}(s)$:

$v = \max(v, \text{min-value}(\text{Successor}))$

return v

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

function min-value(s) returns value

if Terminal-Test(s) then return Utility(s)

initialize $v = +\infty$

for each a in $\text{Actions}(\text{state})$:

$v = \min(v, \text{max-value}(\text{Successor}))$

return v

$$V(s) = \min_{s' \in \text{successors}(s)} V(s')$$

↳ Mini-max search is a Depth first search

Properties:

Time complexity : $O(b^m)$

Space complexity : $O(bm)$

Generalized Minimax

↳ What if the game is NOT zero-sum or has multiple players?

↳ Generalization:

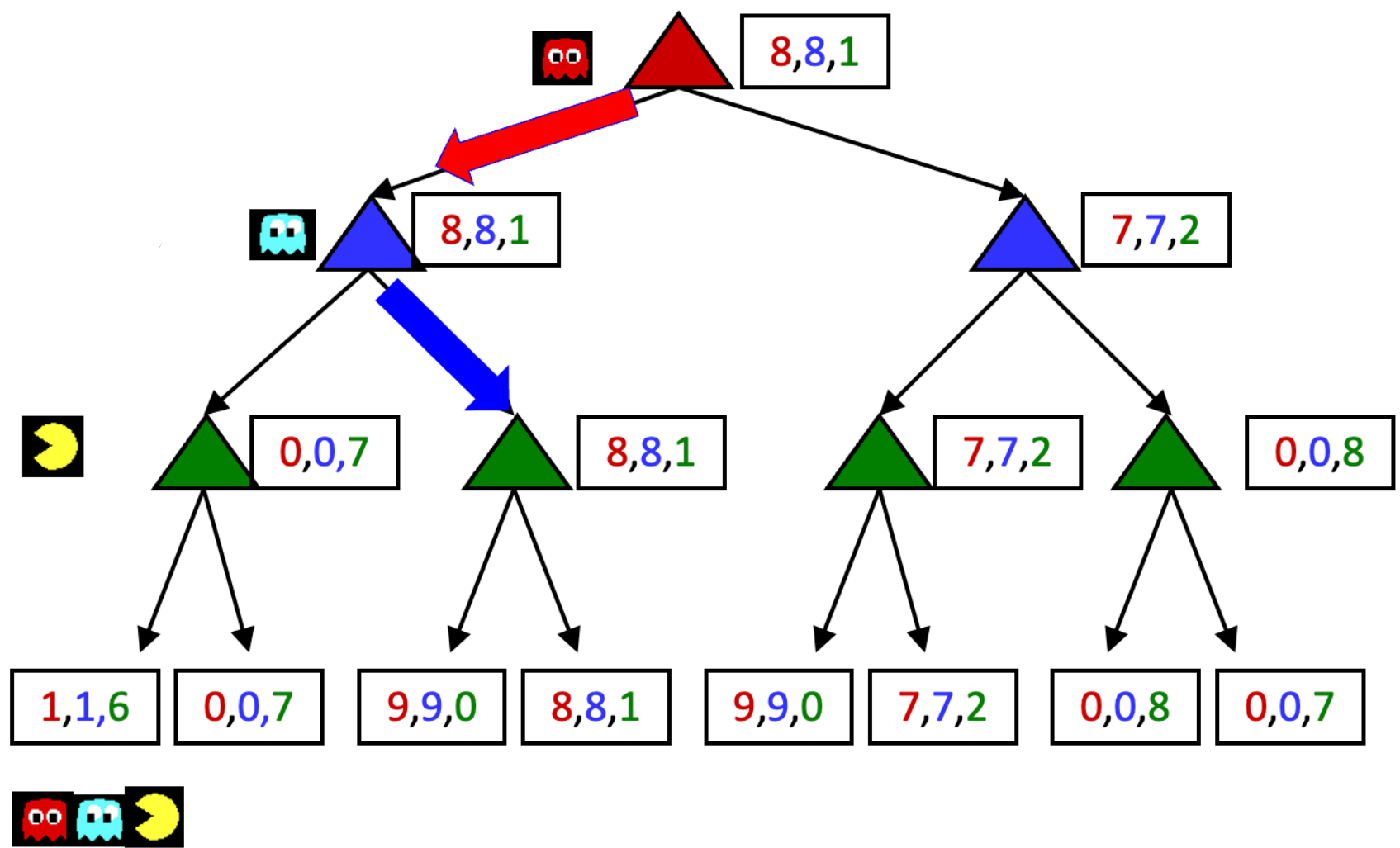
↳ Terminals have utility tuples

↳ Nodes also have utility tuples

↳ Each player maximizes it's own component

↳ Can give rise to cooperation and competition dynamically

Example,



§ If both ghosts go the same way pac-man wins.
If they go different ways pacman loses.

Resource limits:

§ Realistically, we cannot search to leaf nodes.

§ Solution 01:

- Search only to a preset depth limit.
- Use an evaluation function for non-terminal positions.

§ Now, guarantee of optimal play is gone.

§ Evaluation functions are used in depth limited mini-max,

where we treat non-terminal nodes located at our maximum solvable depth as terminal nodes.

Evaluation functions:

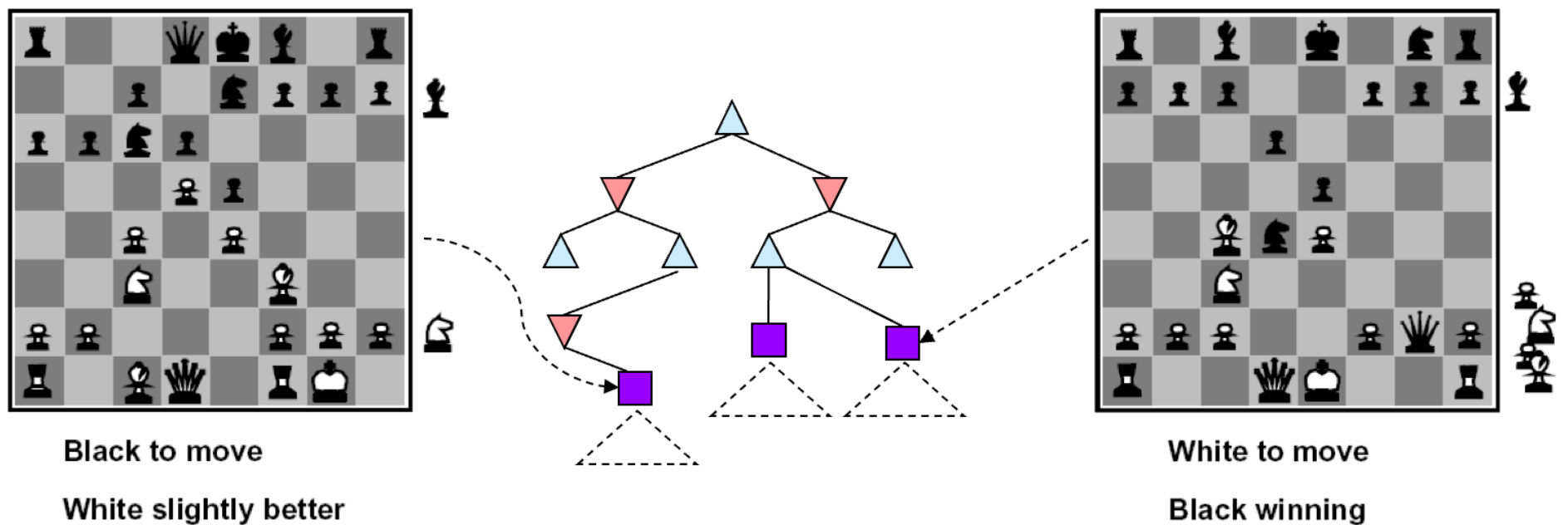
§ Score non-terminals in depth limited search.

§ Ideally, returns the actual minimax value of the position.

↳ In practice, weighted sum of features:

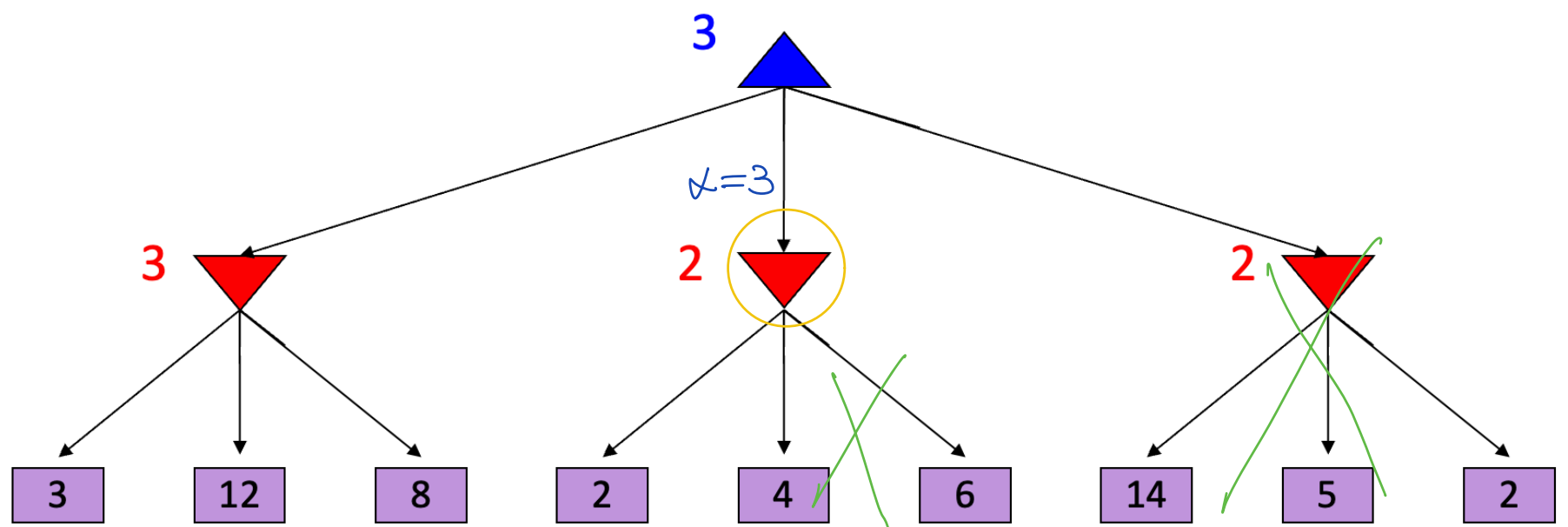
$$EVAL(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

↳ Terminate search only in quiescent positions, i.e. no major changes expected in feature values.



Solution 02: Pruning

↳ Prune branches that can't be chosen



This node chooses the min.
As 2 is value of first child,
we know that the above max node
won't choose this as $3 > 2$. Further,
min node will choose node < 2
but even then max node won't
choose it. So, we prune it.

Alpha-Beta Pruning implementation

We change our min-max value functions as:

```
def max-value(state,  $\alpha$ ,  $\beta$ ):
```

```
    initialize  $v = -\infty$ 
```

```
    for each successor of state:
```

```
         $v = \max(v, \text{min-value}(\text{successor}, \alpha, \beta))$ 
```

```
        if  $v \geq \beta$ 
```

```
            return  $v$ 
```

```
         $\alpha = \max(\alpha, v)$ 
```

```
    return  $v$ 
```

```
def min-value(state,  $\alpha$ ,  $\beta$ ):
```

```
    initialize  $v = +\infty$ 
```

```
    for each successor of state:
```

```
         $v = \min(v, \text{max-value}(\text{successor}, \alpha, \beta))$ 
```

```
        if  $v \leq \alpha$ 
```

```
            return  $v$ 
```

```
         $\beta = \min(\beta, v)$ 
```

```
    return  $v$ 
```

α : MAX's best option on path to root

β : MIN's best option on path to root

Properties:

↳ Has no effect on mini-max value computed for the root.

↳ With "perfect ordering":

Time complexity: $O(b^{n/2})$

Games with uncertain outcomes

Expectations:

↳ The expected value of a random variable is the average, weighted by the probability distribution over outcomes.

Example,

How long to airport

20 min

$\times$

0.25

+

30 min

$\times$

0.50

+

60 min

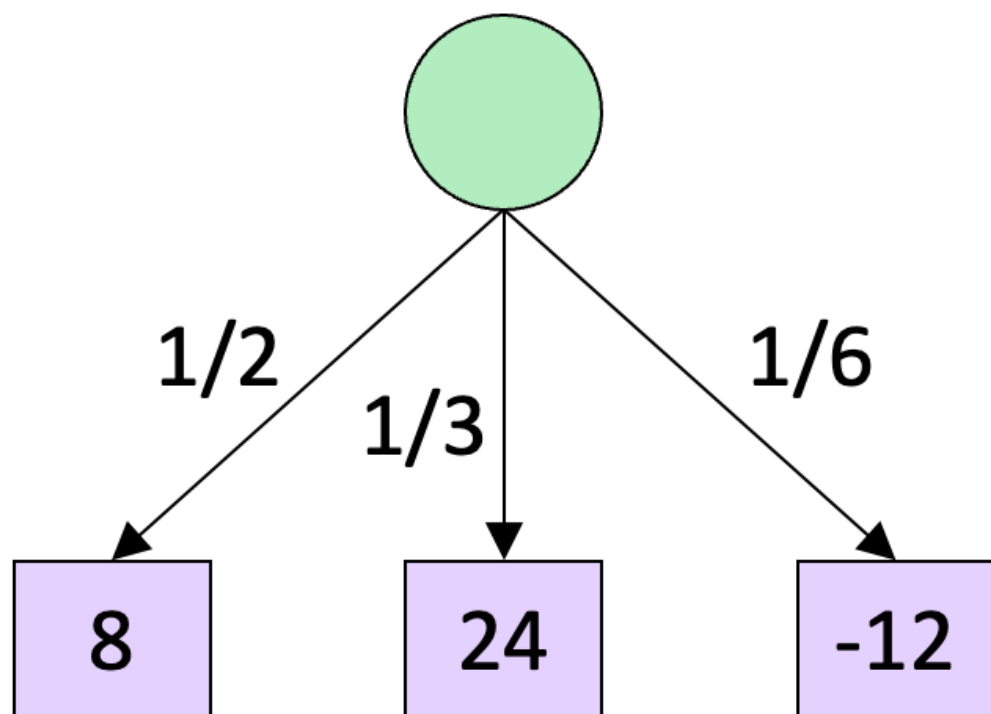
$\times$

0.25

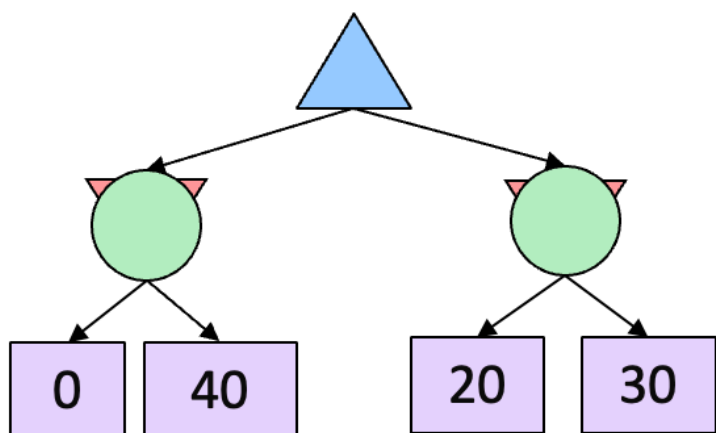
= 35 min

$$v = \frac{1}{2}(8) + \frac{1}{3}(24) + \frac{1}{6}(-12)$$

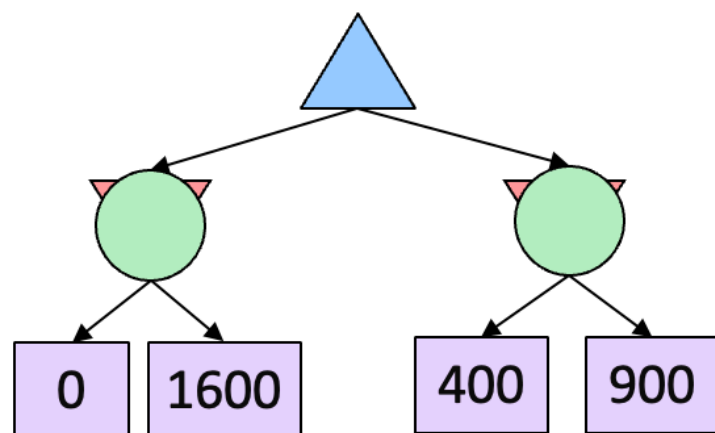
$$= 10$$



What values to use?



x^2



For worst case minimax, evaluation scale doesn't matter.

- We want better states to higher evaluations

- Minimax decisions are invariant to monotonic transformations on values

↳ if $x > y$ then
 $f(x) > f(y)$

↳ Expectimax decisions are invariant to any affine transformations.

↳ $Ax + B, A > 0$